

PROJECT REPORT

Implementation of Client Script & UI Policy on the ServiceNow Incident Form

Project Name	Implement Client Script & UI Policy (Incident)
Platform	ServiceNow (Incident table)
Team Members	Priyankha Sri J, Abinaya R, Monika V
Date	03 October 2026

Table of Contents

1. Introduction

- 1.1 Project Overview
- 1.2 Purpose

2. Ideation Phase

- 2.1 Problem Statement
- 2.2 Empathy Map Canvas
- 2.3 Brainstorming

3. Requirement Analysis

- 3.1 Customer Journey Map
- 3.2 Solution Requirement
- 3.3 Data Flow Diagram
- 3.4 Technology Stack

4. Project Design

- 4.1 Problem Solution Fit
- 4.2 Proposed Solution
- 4.3 Solution Architecture

5. Project Planning & Scheduling

- 5.1 Project Planning

6. Functional and Performance Testing

- 6.1 Performance Testing

7. Results

- 7.1 Output Screenshots

8. Advantages & Disadvantages

9. Conclusion

10. Future Scope

11. Appendix

- Source Code
- Dataset Link
- GitHub & Project Demo Link

1. Introduction

1.1 Project Overview

This project implements client-side behaviour on the ServiceNow Incident form using two platform features: a UI Policy and a Client Script. Both run in the user's browser, so the form reacts instantly to what the user enters, without a round trip to the server.

The work was carried out on a ServiceNow developer instance, on the Incident table [incident]. It has three parts:

- **Part A – UI Policy:** "High Impact Control" makes Assignment group mandatory when an incident has Impact 1 - High.
- **Part B – Client Script:** an onCellEdit script, "Prevent state change via list edit", blocks State changes made by editing a cell in the incident list.
- **Part C – Testing:** the behaviour was tested on new incidents (INC0010001 to INC0010012) and the results were captured as screenshots.

The design also covers two further rules: a UI Policy that shows On hold reason when State is On Hold, and an onChange Client Script that makes Assignment group mandatory for Priority 1 - Critical incidents. These are specified in Sections 3 and 4 and in the Appendix.

1.2 Purpose

The purpose of the project is to control how the Incident form behaves in the browser, and so improve the quality of the data captured, without adding load on the server. Specifically it aims to:

- Make the right fields mandatory, visible or read-only at the right time, using a UI Policy.
- Stop users from changing incident State through quick list editing, using a Client Script.
- Give the team a clear, repeatable step-by-step guide to build, review and test both features.
- Show when to choose a UI Policy and when to choose a Client Script.

2. Ideation Phase

2.1 Problem Statement

On the standard Incident form, important fields can be left empty and users can change the State of an incident from a list view without opening the record. This causes high-impact incidents to be saved without an owning group, and incident state to change without the checks that the full form provides.

Item	Description
Who is affected	ITIL agents / fulfillers who raise and update incidents, and the support managers who rely on the data.
Problem	High-impact incidents can be saved without an Assignment group; State can be changed from the list view, bypassing the form.
Impact of the problem	Delayed assignment, incomplete records, unreliable SLAs and reports.
Desired outcome	The form guides the user at the moment of entry and blocks unwanted shortcuts, with no server round trip.

2.2 Empathy Map Canvas

The empathy map below describes the main user, an ITIL agent working on incidents. It is based on the project scenario.

SAYS / HEARS "I need to log this quickly." "Which group should own this?" Managers ask why critical incidents are unassigned.	THINKS "Is this incident urgent?" "Did I fill in everything required?" Worries about missing a critical ticket.
DOES Opens a new incident, selects Caller, Category and Impact, enters a Short description, edits records directly in the list to save time.	FEELS Pressed for time; frustrated by rework when data is missing; reassured when the form tells them what is needed.

2.3 Brainstorming

The team considered several ways to control the Incident form and chose the approach that gives instant feedback with the least code.

Idea	Where it runs	Decision
Make Assignment group mandatory with a UI Policy when Impact is High	Browser	Selected – no code, reverse logic built in
Block State edits in the list with an onCellEdit Client Script	Browser	Selected – needs a small script
Show / require On hold reason when State is On Hold (UI Policy)	Browser	Selected – declarative
Show a message and require Assignment group when Priority is 1 (onChange Client Script)	Browser	Selected – needs scripting for the message
Validate every rule only in a Business Rule	Server	Kept as back-up only; slower feedback to the user
Use a Data Policy for imports and APIs	Server	Recommended as a complement (see Future Scope)

3. Requirement Analysis

3.1 Customer Journey Map

The journey of an ITIL agent creating a high-impact incident, with the system response at each stage:

Stage	User action	System response (UI Policy / Client Script)	Experience
1. Open	Opens a new Incident (e.g. INC0010005).	Defaults load: Impact 3 - Low, Urgency 3 - Low, Priority 4 - Low. Assignment group is greyed out.	Clear starting form
2. Describe	Selects Caller and Category, types a Short description.	Caller and Short description are mandatory.	Knows what is required
3. Set impact	Sets Impact to 1 - High.	Message "Urgency set to High for High impact incident" appears. Urgency = 1 - High, Priority = 1 - Critical, Assignment group becomes mandatory.	Immediate feedback
4. Assign	Fills Assignment group (CAB Approval) and optionally Assigned to.	Assigned to lookup lists the 6 users.	Confident the ticket is routed
5. Save	Saves from the form menu.	Record is saved with all required data.	Task complete
6. List edit	Tries to change State by editing a cell in the list.	Alert: "State cannot be updated using list editing. Please open the Incident." The edit is cancelled.	Guided to the proper form

3.2 Solution Requirement

Functional requirements

#	Requirement	Mechanism
R1	When State is On Hold, On hold reason is visible and mandatory.	UI Policy
R2	When State is not On Hold, On hold reason is hidden and not mandatory.	UI Policy (Reverse if false)
R3	When Priority is 1 - Critical, Assignment group is mandatory and a message is shown.	Client Script (onChange)
R4	When Priority changes away from 1, the mandatory flag and message are cleared.	Client Script (onChange)
R5	When Impact is 1 - High, Assignment group is mandatory; the requirement is lifted when Impact changes (Reverse if false).	UI Policy "High Impact Control"
R6	State cannot be changed by editing a cell in the incident list; the user is told to open the Incident.	Client Script (onCellEdit)

Non-functional requirements

- Instant response: all checks run in the browser with no server call.
- Configuration is held in the instance and can be moved between instances (Dev, Test, Prod) with an Update Set.
- Rules must not conflict – the same rule is not duplicated in both a UI Policy and a Client Script.

3.3 Data Flow Diagram

The diagram shows how a user action on the Incident form is handled by the UI Policy and Client Script on the client side, and how the data then flows to the server, the Incident table and downstream processes.

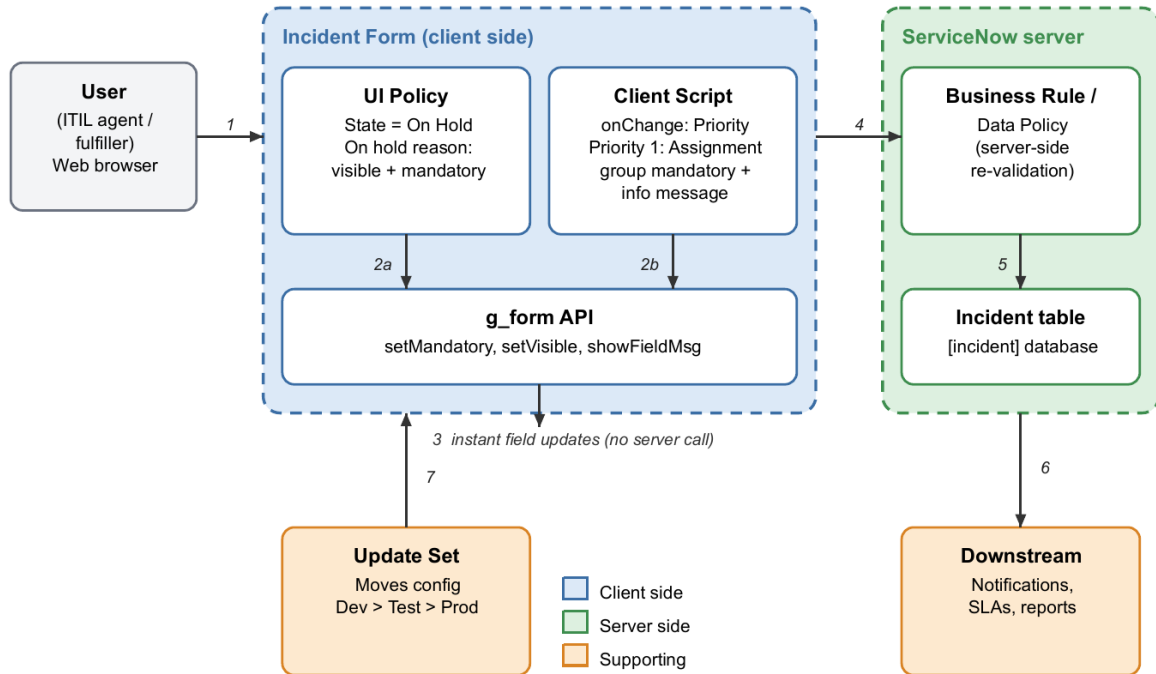


Figure 1: Architecture and data flow of the Incident Client Script and UI Policy implementation

Step	Description
1	The user opens an Incident and changes State, Impact or Priority in the browser.
2a	The UI Policy evaluates its condition (for example Impact is 1 - High, or State is On Hold) and applies its actions to the field.
2b	The onChange / onCellEdit Client Script evaluates the change. For Priority 1, Assignment group becomes mandatory and a message is shown.
3	Both use the g_form API to update the form fields instantly, with no server call.
4	On Save, the form is submitted. A Business Rule or Data Policy re-validates the same rules on the server.
5	Valid data is written to the Incident table.
6	Notifications, SLAs and reports run from the saved record.
7	The configuration is moved between instances (Dev, Test, Prod) with an Update Set.

3.4 Technology Stack

Layer	Technology / Tool
Platform	ServiceNow (Personal Developer Instance), Incident table [incident]
Client-side features	UI Policy and UI Policy Actions; Client Script (onCellEdit, onChange)
Language	JavaScript (client scripts)
Client API	g_form (setMandatory, setVisible, showFieldMsg, hideFieldMsg)

Layer	Technology / Tool
Server-side back-up	Business Rule / Data Policy
Deployment	Update Set (Dev > Test > Prod)
Interface	ServiceNow Platform UI (classic view and Self Service view) in Google Chrome

4. Project Design

4.1 Problem Solution Fit

Problem	Solution	Fit
High-impact incidents saved without an owning group	UI Policy "High Impact Control" makes Assignment group mandatory when Impact is 1 - High	Data is completed at entry time
Fields shown or required when they are not needed	UI Policy with "Reverse if false" restores the normal state automatically	No extra code or manual clean-up
State changed from the list view, bypassing the form	onCellEdit Client Script cancels the edit and informs the user	Edits must go through the full form
Users are unsure why a field is required	Info messages and red asterisks appear instantly	Self-explaining form

4.2 Proposed Solution

Part A – UI Policy: High Impact Control

Navigate to System UI > UI Policies > New and configure:

Field	Value
Table	Incident [incident]
Application	Global
Short description	High Impact Control
Active	True
Order	100
Condition (When to Apply)	Impact is 1 - High
Global	True
On load	True
Reverse if false	True
Inherit	False

Save the policy, then add this UI Policy Action in the related list:

Field name	Mandatory	Visible	Read only
Assignment group	True	Leave alone	False

Part B – Client Script: Prevent state change via list edit

Navigate to System Definition > Client Scripts > New and configure:

Field	Value
Name	Prevent state change via list edit
Table	Incident [incident]
UI Type	Desktop
Type	onCellEdit
Field name	State
Application	Global
Active / Global	True / True
Isolate script	True

```
function onCellEdit(sysIDs, table, oldValues, newValue, callback) {  
    alert('State cannot be updated using list editing. Please open the Incident.');
```

```
    callback(false);  
}
```

Calling callback(false) cancels the cell edit, so the State is not changed from the list.

Additional rules in the design

- **UI Policy "Require On hold reason when State is On Hold":** Condition State is On Hold; On load and Reverse if false True; action sets On hold reason Mandatory True and Visible True.
- **Client Script "Incident - Priority change handler" (onChange on Priority):** makes Assignment group mandatory and shows an info message for Priority 1; clears both otherwise. Full script in the Appendix.

4.3 Solution Architecture

The solution is a two-layer design. The client layer (UI Policy and Client Script) gives instant feedback through the g_form API. The server layer (Business Rule or Data Policy) re-validates on save so that imports and API calls follow the same rules. Configuration is promoted between instances with an Update Set. The architecture is shown in Figure 1 (Section 3.3).

Aspect	UI Policy	Client Script
Best for	Show, hide, mandatory, read-only	Custom logic, messages, calculations
Coding needed	Mostly none (point and click)	JavaScript required
Reverse logic	Built in (Reverse if false)	Must be coded manually
Runs on	Form load and field change	Form load, change, submit or cell edit

5. Project Planning & Scheduling

5.1 Project Planning

The work was done by the three-member team between 1 and 3 October 2026 (dates taken from the screenshots). Task allocation is indicative.

Phase	Activity	Timeline	Output
1. Study	Understand UI Policy, Client Script and g_form; set up developer instance	1 Oct 2026	Working instance
2. Design	Define requirements R1–R6 and architecture	1–2 Oct 2026	Requirement table, diagram
3. Build – Part A	Create UI Policy "High Impact Control" and its action	2 Oct 2026	Steps 1–3
4. Build – Part B	Create onCellEdit Client Script	2 Oct 2026	Steps 6–8
5. Test – Part C	Test on new incidents; capture screenshots	2–3 Oct 2026	Steps 4, 5, 9–17
6. Report	Compile project report	3 Oct 2026	This document

Team members: Priyanka Sri J, Abinaya R, Monika V.

6. Functional and Performance Testing

6.1 Performance Testing

Testing was done manually on the Incident form. Both features run in the browser, so the response is immediate and puts no extra load on the server. The table lists the tests that were executed with screenshot evidence.

ID	Test	Expected result	Actual result	Evidence
F1	Open a new incident	Defaults: Impact 3 - Low, Urgency 3 - Low, Priority 4 - Low; Assignment group greyed out; Caller and Short description mandatory	As expected (INC0010005)	Step 9
F2	Set Impact to 1 - High	Message shown; Urgency 1 - High; Priority 1 - Critical; Assignment group mandatory	As expected (INC0010001, INC0010010, INC0010012)	Steps 4, 11, 15
F3	Fill Assignment group and Short description, then Save	Incident saves with Assignment group CAB Approval	As expected	Steps 11, 15, 17
F4	Assigned to / Caller lookups	Users list opens: 6 users for Assigned to; 638 for Caller	As expected	Steps 12, 13, 16
F5	Impact dropdown options	1 - High, 2 - Medium, 3 - Low	As expected	Step 14
F6	Self Service view (INC0010002)	Urgency read-only, shows 3 - Low	As expected	Step 5
F7	Client Script saved and listed	Script saved with isolate script; appears in Client Scripts list (2,546 scripts)	As expected	Steps 7, 8

Additional test cases defined for the On Hold and Priority rules (R1–R4):

ID	Steps	Expected result
T1	Open a new Incident. Set State to On Hold.	On hold reason appears and is mandatory. Saving without it is blocked.
T2	Change State from On Hold to In Progress.	On hold reason is hidden and no longer mandatory.

ID	Steps	Expected result
T3	Set Priority to 1 - Critical.	Info message appears under Priority. Assignment group becomes mandatory.
T4	Change Priority from 1 to 3.	Message disappears. Assignment group is no longer mandatory.
T5	Open an existing Priority 1 incident.	No message on load (isLoading guard). Form opens normally.

7. Results

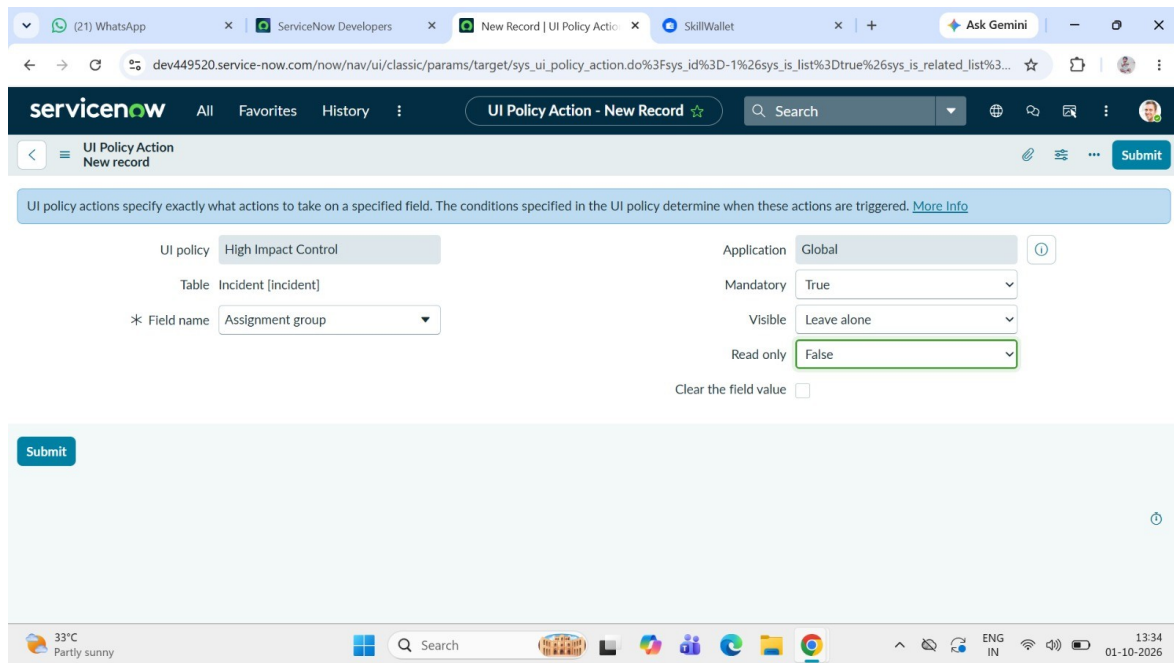
7.1 Output Screenshots

The screenshots below follow the implementation steps in order.

Part A – UI Policy: High Impact Control

The screenshot shows the ServiceNow UI Policy configuration interface. The browser tabs include 'ServiceNow Developers', 'High Impact Control | UI Pol...', and 'SkillWallet'. The URL is 'dev449520.service-now.com/now/nav/ui/classic/params/target/sys_ui_policy.do%3Fsys_id%3Daf06ef9b83ab8310a0a6e590cead34e%26sysparm_view...'. The page title is 'UI Policy - High Impact Control'. The configuration is for a 'High Impact Control' policy, which is 'Active'. The 'Short description' is 'High Impact Control' and the 'Order' is '100'. The 'When to Apply' tab is selected, showing conditions: 'Impact is 1 - High'. The policy is configured to be 'Global', 'On load', and 'Reverse if false'. The 'Script' tab is also visible, showing the policy's logic: 'If selected, the UI Policy applies to all form views; otherwise the UI Policy is view-specific'. The 'Inherit' checkbox is unchecked.

Figure 2: Step 1 – UI Policy conditions: Impact is 1 - High; Global, On load and Reverse if false ticked.



UI policy actions specify exactly what actions to take on a specified field. The conditions specified in the UI policy determine when these actions are triggered. [More Info](#)

UI policy: High Impact Control

Table: Incident [Incident]

* Field name: Assignment group

Application: Global

Mandatory: True

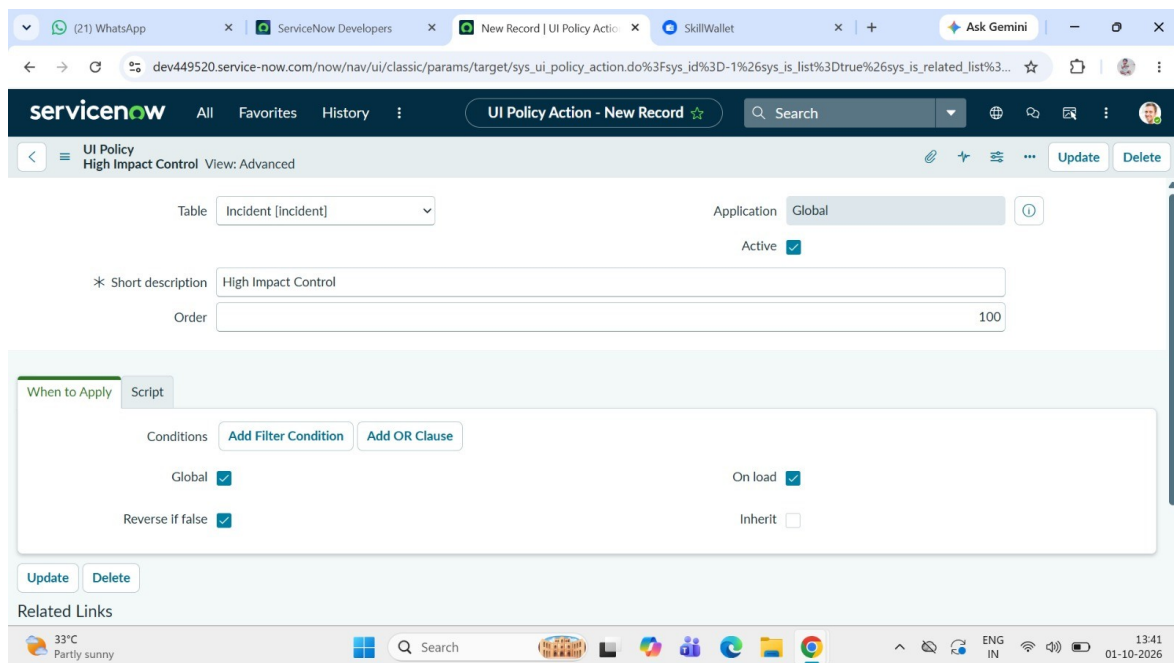
Visible: Leave alone

Read only: False

Clear the field value: ☐

Submit

Figure 3: Step 2 – UI Policy Action: Assignment group, Mandatory True, Visible Leave alone, Read only False.



UI Policy Action - New Record

Table: Incident [Incident]

Application: Global

Active: ☒

* Short description: High Impact Control

Order: 100

When to Apply

Conditions: Add Filter Condition Add OR Clause

Global: ☒

Reverse if false: ☒

On load: ☒

Inherit: ☐

Update Delete

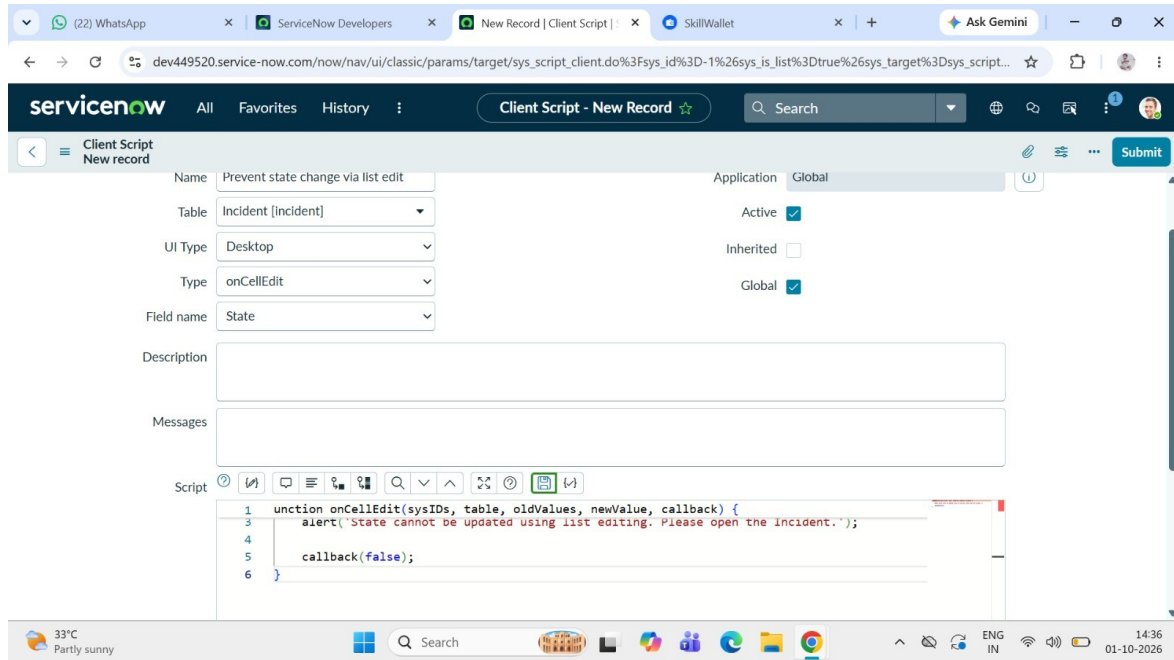
Related Links

Figure 4: Step 3 – UI Policy settings: Table Incident, Application Global, Short description High Impact Control, Order 100.

Figure 5: Step 4 – Test: Impact 1 - High gives the message, Urgency 1 - High, Priority 1 - Critical and mandatory Assignment group.

Figure 6: Step 5 – Self Service view: Urgency is read-only and shows 3 - Low.

Part B – Client Script: Prevent state change via list edit



The screenshot shows the 'Client Script - New Record' form in ServiceNow. The form fields are as follows:

Field	Value
Name	Prevent state change via list edit
Table	Incident [Incident]
UI Type	Desktop
Type	onCellEdit
Field name	State
Application	Global
Active	☒
Inherited	☐
Global	☒

Description: [Empty text area]

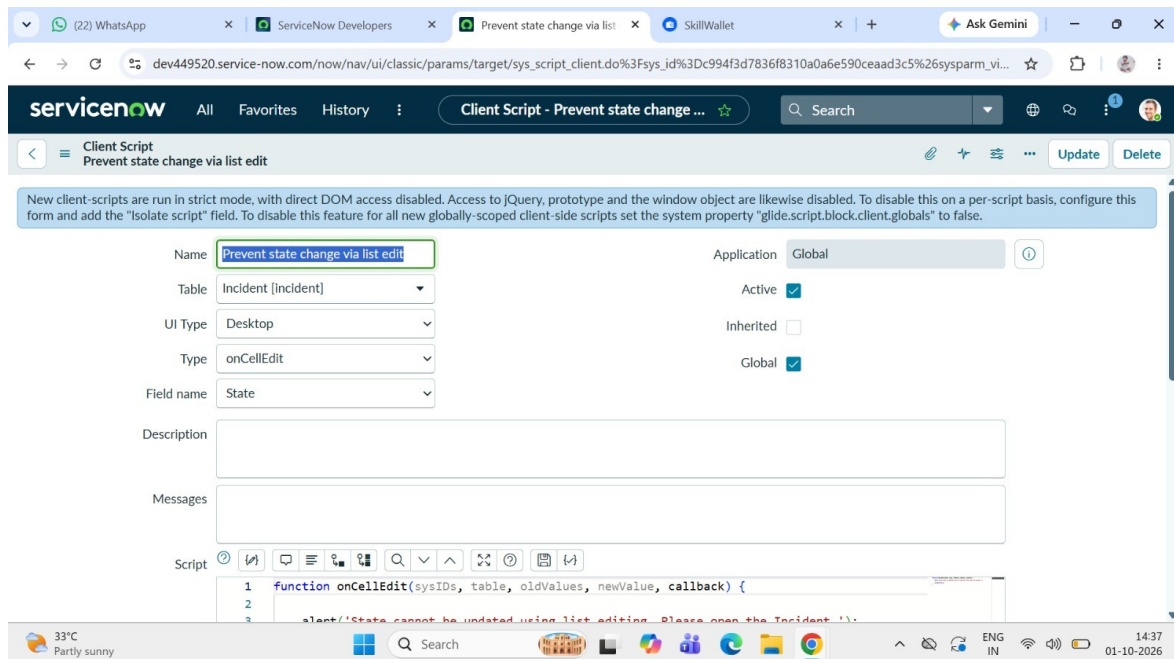
Messages: [Empty text area]

Script: [Code editor with the following code]

```
1 unction onCellEdit(sysIDs, table, oldValues, newValue, callback) {  
2     alert("State cannot be updated using list editing. Please open the Incident.");  
3  
4     callback(false);  
5  
6 }
```

Figure 7: Step 6 – New Client Script: Table Incident, UI Type Desktop, Type onCellEdit, Field name State.

Note: the first word of the script reads "unction" in this screenshot; it was corrected to "function" before saving (see the next figures).



The screenshot shows the 'Client Script - Prevent state change via list edit' form in ServiceNow. The form fields are as follows:

Field	Value
Name	Prevent state change via list edit
Table	Incident [Incident]
UI Type	Desktop
Type	onCellEdit
Field name	State
Application	Global
Active	☒
Inherited	☐
Global	☒

Description: [Empty text area]

Messages: [Empty text area]

Script: [Code editor with the following code]

```
1 function onCellEdit(sysIDs, table, oldValues, newValue, callback) {  
2  
3     alert("State cannot be updated using list editing. Please open the Incident.");  
4  
5 }
```

Figure 8: Step 7 – Saved Client Script record with Update and Delete buttons and Isolate script ticked.

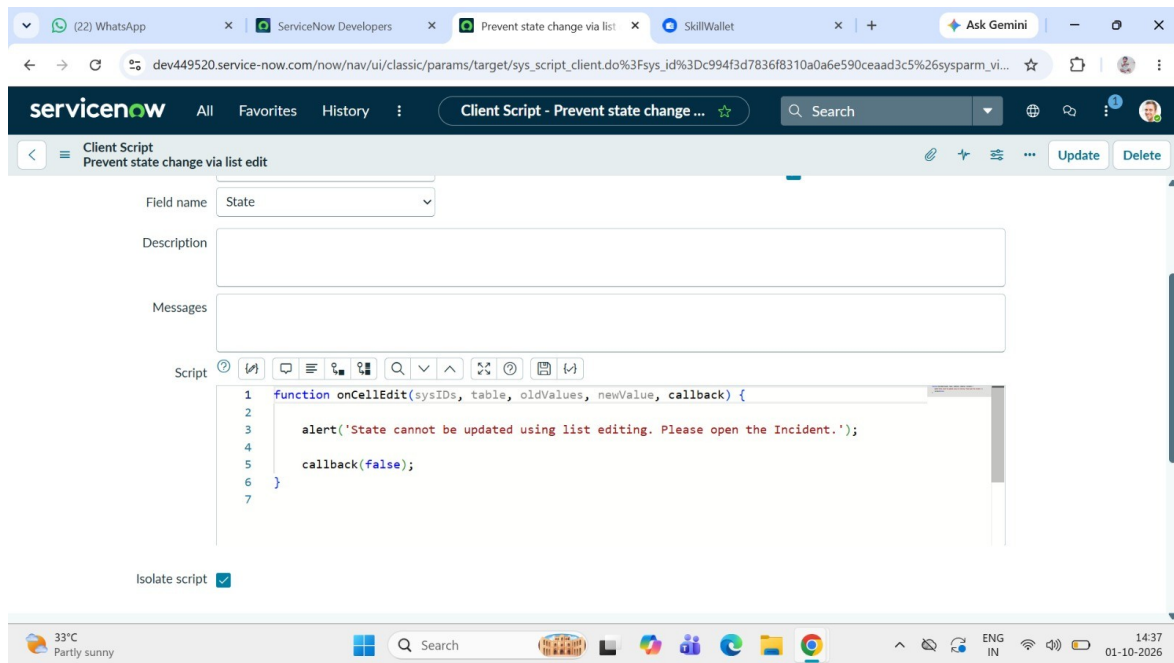


Figure 9: Step 7 – Script body after the correction.

The screenshot shows the ServiceNow Client Scripts list. The table contains the following data:

Name	Active	Table	Application	View	Type	Updated
'Enable related search box' Restriction	true	Table Configuration [cxs_table_config]	Global		onChange	2025-09-03 13:00:30
'Scan Source' = 'Manual' for new records	true	Payload [sys_app_scan_payload]	Global		onLoad	2021-12-03 07:05:37
'Selection Required' label for Checkbox	true	Variable [item_option_new]	Global		onLoad	2018-08-17 14:18:47
'Selection Required' label for Checkbox	true	Variable [item_option_new]	Global		onChange	2018-08-17 14:18:47
(BP) Close Mandatory on Close or Resolve	true	Incident [incident]	Global		onSubmit	2022-11-05 23:45:46
(BP) Hide Attachment Link when Closed	true	Incident [incident]	Global		onLoad	2018-02-28 02:27:03
(BP) Hide Choice - Closed	true	Incident [incident]	Global		onLoad	2025-04-18 11:39:13
(BP) Hide Close Notes and Code	true	Incident [incident]	Global		onLoad	2018-01-25 03:16:44

The browser's address bar shows the URL: `dev449520.service-now.com/now/nav/ui/classic/params/target/sys_script_client_list.do%3Fsysparm_query%3Dsys_class_name%253Dsys_script_client%...`

Figure 10: Step 8 – Client Scripts list (1 to 20 of 2,546), used to find the new script by Name.

Part C – Testing on the Incident form

The screenshot shows the ServiceNow Incident form for INC0010005. The form is titled "Incident - Create INC0010005". The "Number" field is populated with "INC0010005". The "Channel" is set to "-- None --". The "State" is set to "New". The "Impact" is set to "3 - Low". The "Urgency" is set to "3 - Low". The "Priority" is set to "4 - Low". The "Assignment group" field is greyed out. The "Assigned to" field is empty. The "Short description" and "Description" fields are empty. The "Related Search Results" section shows "No results to display".

Figure 11: Step 9 – New incident INC0010005: default Impact, Urgency and Priority all Low; Assignment group greyed out.

The screenshot shows the ServiceNow Incident form for INC0010005. The "incident.form" search is active in the application navigator. The search results show "No Results" for both "FAVORITES" and "ALL RESULTS". The form fields are the same as in Figure 11. The "Related Search Results" section shows "No results to display".

Figure 12: Step 10 – Application navigator search for "incident.form": Favorites and All Results show No Results.

Incident - Create INC0010010

Number: INC0010010

* Caller: Abel Tutor

Category: Inquiry / Help

Subcategory: -- None --

Service:

Service offering:

Configuration item:

* Short description: Hii

Description:

Channel: -- None --

State: New

Impact: 1 - High

Urgency: 1 - High

Priority: 1 - Critical

* Assignment group: CAB Approval

Assigned to:

Related Search: Hii

No matching results found for Hii

Figure 13: Step 11 – INC0010010: Impact 1 - High; Urgency 1 - High, Priority 1 - Critical, Assignment group CAB Approval (mandatory).

Incident - Create INC0010010

Number: INC0010010

* Caller: Abel Tutor

Category: Inquiry / Help

Subcategory: -- None --

Service:

Service offering:

Configuration item:

* Short description: Hii

Description:

Channel: -- None --

State: New

Impact: 1 - High

Urgency: 1 - High

Priority: 1 - Critical

* Assignment group: CAB Approval

Assigned to:

Related Search: Hii

Users | ServiceNow - Google Chrome

dev449520.service-now.com/sys_user_list.do?sysparm_target=incident.assigned_to&sysparm_target_value=8&sysparm_f...

Users

Name	First name	Last name	Email
Bernard Laboy	Bernard	Laboy	bernard.laboy@example.com
cab approver	cab	approver	cab.approver@example.com
Christen Mitchell	Christen	Mitchell	christen.mitchell@example.com
Howard Johnson	Howard	Johnson	howard.johnson@example.com
Luke Wilson	Luke	Wilson	luke.wilson@example.com
Ron Kettering	Ron	Kettering	ron.kettering@example.com

1 to 6 of 6

Figure 14: Step 12 – Assigned to lookup: Users list with 6 users.

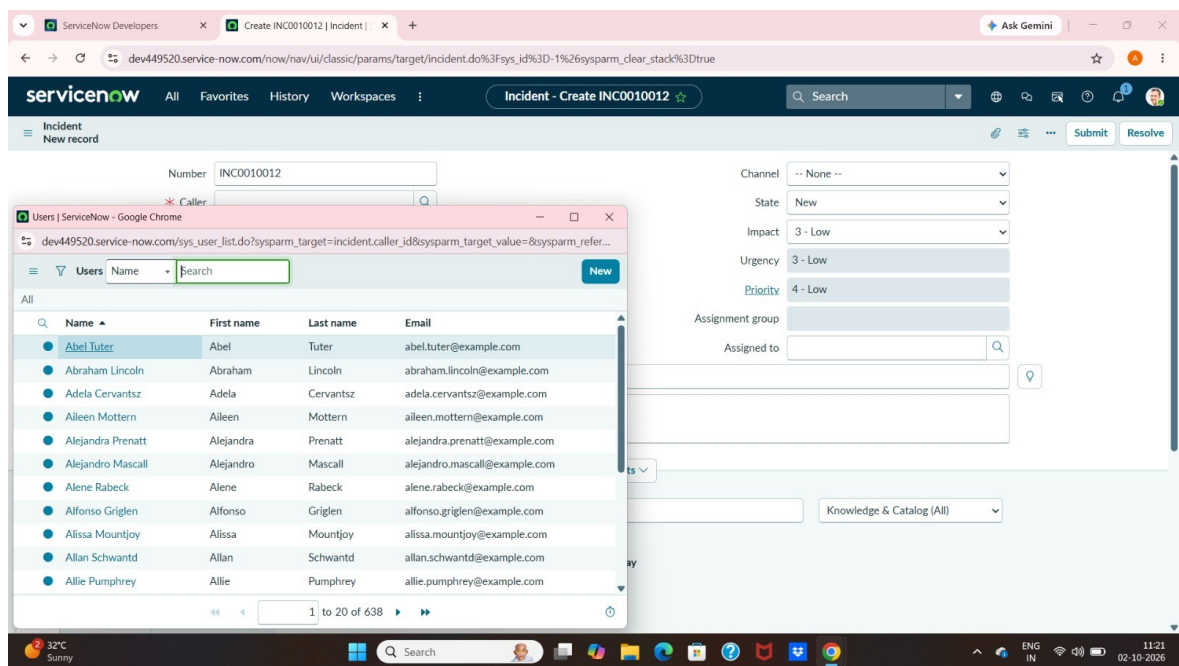


Figure 15: Step 13 – INC0010012: Caller lookup (1 to 20 of 638), Abel Tuter selected.

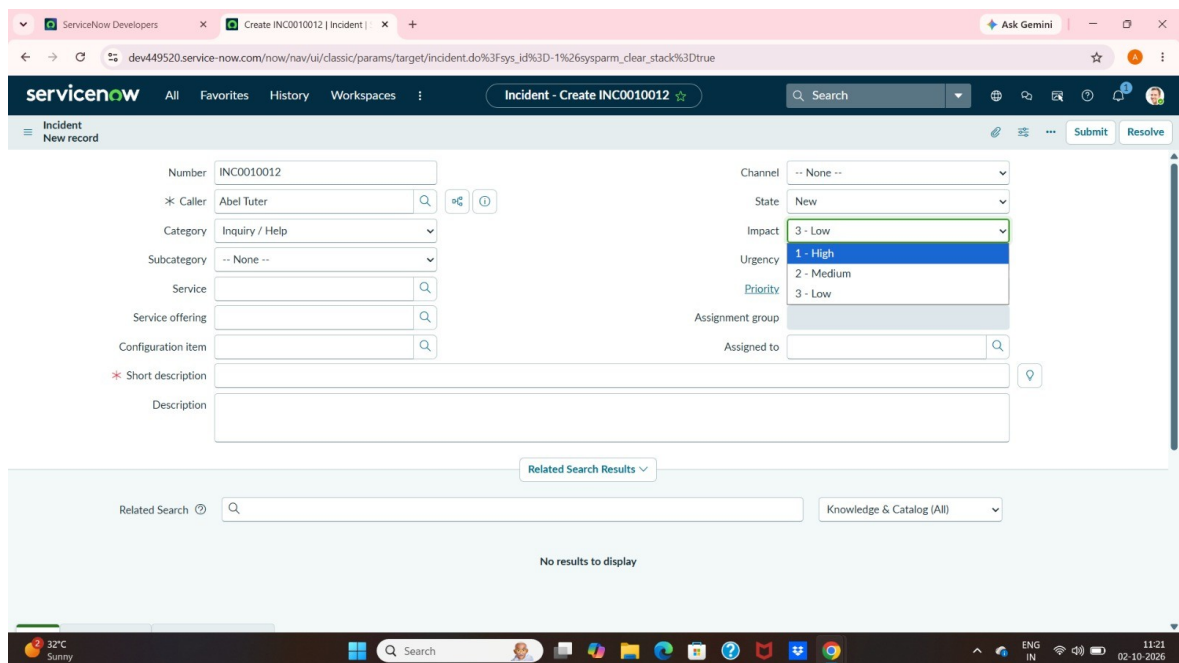


Figure 16: Step 14 – Impact dropdown: 1 - High, 2 - Medium, 3 - Low.

Incident - Create INC0010012

Additional actions: Urgency set to High for High impact incident.

Number: INC0010012

* Caller: Abel Tutor

Category: Inquiry / Help

Subcategory: -- None --

Service:

Service offering:

Configuration item:

* Short description: hii

Description:

Channel: -- None --

State: New

Impact: 1 - High

Urgency: 1 - High

Priority: 1 - Critical

* Assignment group: CAB Approval

Assigned to:

Related Search Results

Related Search:

Knowledge & Catalog (All)

No results to display

Figure 17: Step 15 – Impact 1 - High: Urgency 1 - High, Priority 1 - Critical, mandatory Assignment group; Short description entered.

Incident - Create INC0010012

Additional actions: Urgency set to High for High impact incident.

Number: INC0010012

* Caller: Abel Tutor

Category: Inquiry / Help

Subcategory: -- None --

Service:

Service offering:

Configuration item:

* Short description: hii

Description:

Channel: -- None --

State: New

Impact: 1 - High

Urgency: 1 - High

Priority: 1 - Critical

* Assignment group: CAB Approval

Assigned to:

Related Search Results

Related Search: hii

Users | ServiceNow - Google Chrome

dev449520.service-now.com/sys_user_list.do?sysparm_target=incident.assigned_to&sysparm_target_value=&sysparm_f...

Users: Name Search

All

Name	First name	Last name	Email
Bernard Laboy	Bernard	Laboy	bernard.laboy@example.com
cab approver	cab	approver	cab.approver@example.com
Christen Mitchell	Christen	Mitchell	christen.mitchell@example.com
Howard Johnson	Howard	Johnson	howard.johnson@example.com
Luke Wilson	Luke	Wilson	luke.wilson@example.com
Ron Kettering	Ron	Kettering	ron.kettering@example.com

1 to 6 of 6

Figure 18: Step 16 – Assigned to lookup again: 6 of 6 users.

Figure 19: Step 17 – Form menu with Save, Configure, Export, Create Favorite, Copy URL, Copy sys_id and Reload form.

8. Advantages & Disadvantages

Advantages	Disadvantages
Instant feedback in the browser with no server round trip.	Client-side rules only affect the browser form; imports and APIs can bypass them.
UI Policy needs little or no code and has built-in "Reverse if false".	Client Scripts need JavaScript and reverse logic must be coded by hand.
Improves data quality, e.g. high-impact incidents always get an Assignment group.	A UI Policy and a Client Script on the same rule can conflict.
onCellEdit script keeps State changes on the full form.	An alert() blocks the user until dismissed; a field message is less intrusive.
Easy to move between instances with an Update Set.	Many global scripts (2,546 in the list) make naming and ordering important.

9. Conclusion

The project shows that a UI Policy and a Client Script can control the Incident form without any server impact. The "High Impact Control" UI Policy makes Assignment group mandatory when Impact is High (and Urgency and Priority follow), and the onCellEdit Client Script stops State being changed from the list view. Testing on several new incidents confirmed the expected behaviour. The designed On Hold and Priority rules extend the same approach. Together they improve data quality, guide users while they work, and add no load on the server.

- Prefer a UI Policy for simple visibility and mandatory rules; use a Client Script only when scripting is needed.
- Back critical rules with a Business Rule or Data Policy so imports and APIs obey them too.
- Always include the isLoading check in onChange scripts.
- Test in a sub-production instance and capture the changes in an Update Set.

- Avoid duplicating the same rule in both a UI Policy and a Client Script.

10. Future Scope

- Add a Business Rule or Data Policy so that the same rules apply to imports, integrations and REST API calls.
- Replace the alert() in the onCellEdit script with a non-blocking message, and allow exceptions for admin roles.
- Extend the rules to other fields and tables (for example Change and Problem).
- Add automated tests (ServiceNow Automated Test Framework) for the form behaviour.
- Apply the rules in Workspace and the Service Portal using UI Type = All.
- Package the configuration in a scoped application for easy reuse.

11. Appendix

Source Code

Client Script – Prevent state change via list edit (onCellEdit on State)

```
function onCellEdit(sysIDs, table, oldValues, newValue, callback) {
    alert('State cannot be updated using list editing. Please open the Incident.');
```

```
    callback(false);
}
```

Client Script – Incident - Priority change handler (onChange on Priority)

```
function onChange(control, oldValue, newValue, isLoading, isTemplate) {
    if (isLoading || newValue === '') {
        return;
    }

    // Priority 1 - Critical
    if (newValue === '1') {
        g_form.setMandatory('assignment_group', true);
        g_form.showFieldMsg('priority', 'Critical incident: assign a group
immediately.', 'info');
    } else {
        g_form.setMandatory('assignment_group', false);
        g_form.hideFieldMsg('priority', true);
    }
}
```

How it works:

- isLoading is true while the form first loads; returning early avoids firing the logic before the user changes anything.
- g_form.setMandatory() toggles the red asterisk and blocks save when the field is empty.
- g_form.showFieldMsg() / hideFieldMsg() display or clear an inline message under the Priority field.

UI Policy – Require On hold reason when State is On Hold

Setting	Value
Table	Incident [incident]
Conditions	State is On Hold
Active / On load / Reverse if false	True / True / True
Action: On hold reason	Mandatory True, Visible True, Read only Leave alone

Setting	Value
Action: Resolution notes (optional)	Mandatory Leave alone, Visible False, Read only Leave alone

Dataset Link

Not applicable. No external dataset was used; testing used sample records on the ServiceNow developer instance (incidents INC0010001 to INC0010012).

GitHub & Project Demo Link

GitHub repository: <https://github.com/moniscode/Service-now-project>

Project demo link: <https://drive.google.com/drive/folders/1GreXkfCRr86m4Z5qc8s5UvoWbxQcAJ5y>